Grammar-Aware Fuzzing of PostgreSQL with Typed Derivation Trees, and Beyond

CS-5187 Software Analysis, Term Project A3 final presentation

Venu Kummari Bhuvan Chilukuti Gowtham Reddy

November 2026

IIT Guwahati

The problem: bugs that do not crash

- Target: **PostgreSQL**, the most used open-source database; its planner changes weekly.
- Worst bug class: a **wrong result**: no error, no crash, no log line.
- Crash fuzzers (SQLsmith, Squirrel) cannot see it; SQLancer finds it with small hand-written generators.
- Both need a **test oracle**: how do we know the right answer?

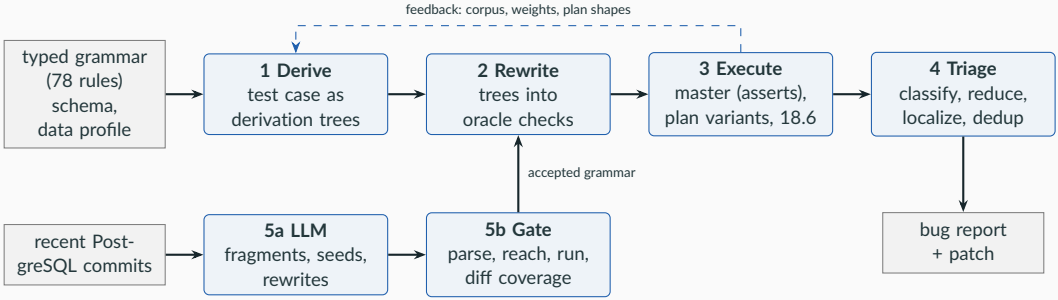
Our answer
Generate SQL as typed derivation trees and turn each tree into checks that know the answer: equivalent queries, other plans, the last release.

Outline

1. **Arbor**, the PostgreSQL fuzzer: architecture and five techniques
2. Evaluation and **how the tool found each bug**
3. The **generalized fuzzer**: DuckDB, SQLite, Turso, DataFusion
4. Team, limitations, conclusion

Arbor: the PostgreSQL fuzzer

Architecture



- Python, about 10,500 lines, 151 unit tests; one worker per server, savepoints isolate test cases.
- The derivation tree is used four times: generate, mutate, rewrite into oracles, reduce.

Technique 1: a typed grammar that produces trees

- Own DSL (`.pgg`): 78 rules, 421 alternatives, 13 SQL types; types are parameters (`expr<T>`).
- Scope-aware: names, aliases, column types bound while deriving ⇒ 99.3% of statements valid (random mutation: 26%).
- Each test case is kept as trees; **replay** re-derives a tree after an edit and re-binds names.
- Feedback: corpus of novel test cases, subtree mutation, bandit weights per alternative, data-profile literals near real values.

```
expr<T:num> : expr<T> "+" expr<T>
            | <0.5> ABS "(" expr<T> ")"
            | @col(T) | @lit(T) ;
pred : expr<T> cmp expr<T>
     | expr<T> IS NOT? NULL ;
```

Types make every subtree replaceable by another subtree of the same type: the basis of the rewrites and of reduction.

Technique 2: the oracle stack

Oracle	What is compared or detected	Bug class
crash / assertion	backend dies; assertion text from the server log	crash
internal error	SQLSTATE `XX000` (“should not happen”)	internal error
hang	query outlives `statement_timeout` and cancel	hang
plan variants	same query under 18 planner settings (join methods, parallel, …)	wrong result
metamorphic	query vs. its tree rewrite: TLP, NoREC, expression, predicate	wrong result
cross-version	master vs. release 18.6, same database	regression
CERT	adding a restriction must not raise the row estimate	bad estimate
RLS	row-level security must hide the same rows as a filter	data leak

Determinism filter: only statements whose results may be compared (no `LIMIT` without total order, no clock functions); floats masked once per check.

Technique 3: metamorphic rewrites on the tree

TLP (ternary logic partitioning): pick a predicate p in the tree, then

$$Q \;=\; Q_p \uplus Q_{\neg p} \uplus Q_{p\ \mathrm{IS\ NULL}}$$

NoREC: the filter moved into the select list must count the same rows.
Expression rewrites: $e \to (\mathrm{SELECT}\ e)$, $e + 0$, … for a typed subtree.

- Applied at any query level (subqueries, CTEs), because the tree tells where a predicate is and what type it has.
- A mismatch is **confirmed** by re-running both sides before it is reported.
- 12 rewrite kinds; each was checked on live runs for false positives.

Technique 4: triage, from a failure to a report

1. **Reproduce** 3 times on a fresh server (else flaky).
2. **Reduce**, three levels:
   - tree: replace a subtree by the smallest derivation of the same rule and type, replay;
   - statement delta debugging (ddmin);
   - token hierarchical delta debugging.
3. **Localize**: which plan variant, rewrite or version flips it.
4. **Deduplicate** by signature (outcome, culprit, stack frames).

Why the tree helps

Token reduction gets stuck when removing any token breaks the SQL. Tree reduction keeps every candidate valid: **116** $\rightarrow$ **79 tokens** on a real case.

- Verdict ledger: 93 candidates, every one decided.
- Git bisection script for regressions.

Technique 5: the LLM proposes, the server decides

1. **Mine** recent planner/executor commits (+ their regression tests).
2. **Prompt**: commit diff + grammar excerpt $\rightarrow$ grammar fragment, seed tests, rewrite templates.
3. Every prompt and response stored; replay is deterministic and free.

Validation gate (on the server)

1. parses with the grammar, known builtins only
2. reachable: derived in ≥ 5 of 150 samples
3. $\geq 60\%$ of its statements run
4. seeds run
5. executes more of the commit's changed lines (gcov diff coverage)

Rewrites also need a value probe + 150 live checks.

Evaluation and how the bugs were found

Evaluation setup and results

Base: master `26bd08fb` with assertions; reference 18.6; **5 trials** $\times$ **1 h** per arm; Mann–Whitney U, Vargha–Delaney $\hat{A}_{12}$.

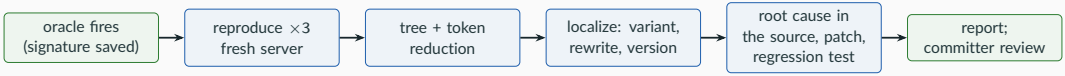
RQ	Result
RQ1 reach	valid statements 99.3% vs 26.0% (random); backend lines **36.7% vs 22.2%** ($\hat{A}_{12} = 1.00$, $p = 0.008$); oracles: plan shapes $\times 1.7$
RQ2 bugs	93 candidates: 8 real, 15 duplicates of real bugs, 42 false positives (all classes fixed), 28 flaky
RQ3 benchmark	23 real fixes reverted in one build; best arm found 2 of 20 in 1 h $\Rightarrow$ 1 h is short
RQ4 LLM	+5 SQL features ($p = 0.004$), no coverage gain; 4 of 10 commits pass the gate

Lesson: oracle precision, not generator reach, limits a wrong-result fuzzer.

How the tool found the PostgreSQL bugs

Bug type	Technique that found it	How the signal appeared	Upstream
assertion (parallel plans)	LLM fragment + TLP check	assertion fired while running the TLP check of an LLM-derived query; text taken from the server log	**fixed** #19803
wrong output (formatting)	cross-version oracle	master and 18.6 printed different strings for the same call	**fixed, our patch** #19771
assertion (statistics)	extended-statistics clause generator	random statistics objects + clauses reusing their expressions	**fixed, our patch**
assertion / wrong result	grammar fuzzer, then cross-version	assertion on master; 18.6 gave a silently wrong answer	**fixed, our patch** #19768
wrong result (anti-join)	cross-version + TLP	master returned a row that 18.6 and the partition sum did not	**fixed, our patch**
hang	hang oracle	query outlived the timeout and ignored cancel	**fixed, our patch**
internal error	internal-error oracle (`XX000`)	grammar arm, `MERGE` on partitioned tables	**confirmed** #19802
wrong result (constant folding)	expression rewrite $e \to (\mathrm{SELECT}\ e)$	the sub-select blocked a planner simplification; results differed	**open** #19793
bad estimate	CERT oracle	a restriction raised the row estimate (1000 $\to 10^{100}$)	**open** (patch)

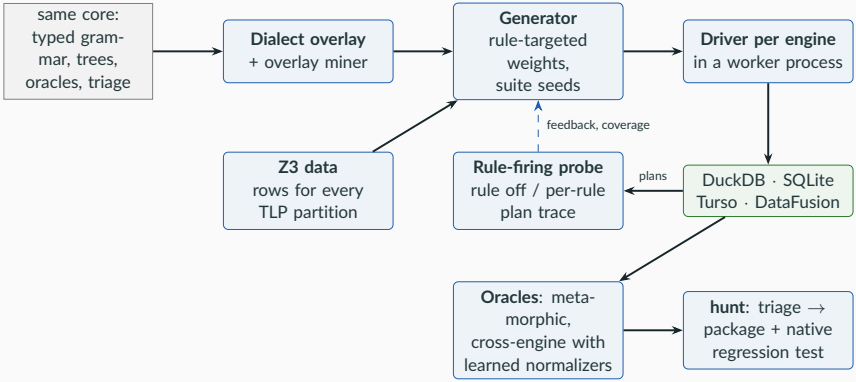
From signal to accepted fix: one pipeline for every bug



- Example: the cross-version oracle flagged a difference; reduction left one function call; localization: “version”; the root cause: the string terminator was copied into the output; our patch was committed.
- **Result: 9 reported, 6 fixed upstream (5 with our patches), 1 confirmed.**
- One command reproduces the headline: `sudo bash scripts/reproduce.sh` ⇒ “Known bugs reproduced: 9 of 9”.

The generalized fuzzer

Beyond PostgreSQL: architecture



- Same trees and oracles; new: drivers, overlays, rule firing, Z3 data, learned normalizers, autonomous loop.
- Branch `generalized`, 192 unit tests.

Generalized techniques (1): engines, overlays, rule firing

Engine abstraction: `Driver` (execute, error classes, rules, plans) in a spawned worker: a crash or Rust panic kills only the worker.

Dialect overlays: `%drop`, `%replace`, `%droptype` with cascade over the PostgreSQL grammar. **Overlay miner**: alternatives that almost never execute become `%drop` lines: DuckDB **32%** → **44%** valid with no manual line.

Valid queries with overlays: DuckDB 96.3%, SQLite 97.7%, Turso 93.3%, DataFusion 79.4%.

Optimizer-rule firing replaces code coverage:

- rule fires ⇔ plan changes when the rule is switched off;
- SQLite: 30 switches via `sqlite3_test_control` (found in the symbol table); DuckDB: `disabled_optimizers`; DataFusion: `EXPLAIN VERBOSE` shows each rule's effect;
- uses: feedback (new rules/pairs), coverage metric, rule-targeted generation, localization;
- ground truth: SQLite gcov build, 69.4% of optimizer lines from 200 cases.

Generalized techniques (2): data, comparison, autonomy

- **Z3 data synthesis**: the predicate subtree (comparisons, `AND/OR/NOT`, NULL logic, `BETWEEN`, `IN`, `LIKE`) becomes constraints; Z3 yields rows so every TLP partition is non-empty (non-vacuous checks 34% → 40%, first version).
- **Learned normalizers**: probe two engines with small typed queries, cluster disagreements, pick normalizers (float digits, `2.0/2`, booleans) and guards: DuckDB vs SQLite disagreements **28.7%** → **3.6%**.
- **Test suites as seeds**: sqllogictest and Turso tests imported (DataFusion: 406 files, 19,602 statements, 83% run).
- **Autonomous loop** `pgfuzz hunt`: campaign → triage → package with a regression test in the engine's own format (`.test`, `.slt`, `.sqltest`).

How the generalized fuzzer found its bugs

Engine	Bug type	How it was found	State
Turso	wrong result / spurious error	Turso-vs-SQLite differential oracle; reduced to one aggregate call; root cause: arguments read from non-consecutive registers	**filed with fix** #11594 / PR #11595
DuckDB	internal error (2.0.0 regression)	internal-error oracle on the nightly build; a regression since 2.0.0; root cause: the binder copies an expression that cannot be copied	**filed with fix** #28551 / PR #28552
DataFusion	internal error in DML	internal-error oracle; root cause: common-subexpression elimination leaks a plan-only column into DML planning	**filed with fix** #27235 / PR #27236
DataFusion	internal error (planning order)	internal-error oracle	fix ready
Turso, DuckDB	panic, rejected join, bind error	worker caught a Rust panic / error; bisection on `main`	fixed upstream before we filed

Every filed fix has a regression test in the engine's own format and passes the engine's full test suite (e.g. 12,700 Turso tests, all 546 DataFusion sqllogictest files).

Wrap-up

Who did what

Venu Kummari

- tool design and implementation: grammar, oracles, triage, LLM gate, benchmark
- paper; #19768, #19793
- generalized fuzzer and its 3 fixes

Bhuvan Chilukuti

- RQ3 benchmark campaign
- bug hunts on recent features
- patches: NOT IN, IGNORE NULLS, statistics, MCV

Gowtham Reddy

- server builds, RQ1/RQ2/RQ4 campaigns
- LLM arm (10 commits)
- #19802, #19803; `to_char` fix

Signed statement in the paper; commit history shows the same split. Each of us can explain every part.

Limitations and future work

- 5 trials per arm: effect sizes rather than always significance; false-positive fixes not re-measured by a new campaign.
- One schema, one commit for PostgreSQL; LLM: free-tier model on ten commits.
- Generalized: evaluation runs (rule feedback, Z3, normalizers, SQLancer baseline, DuckDB benchmark) are scripted and dry-run tested; the full runs take about four days.
- DataFusion validity (79%) below the 95% of the other engines.

Conclusion

9
PostgreSQL bugs reported

6
fixed upstream (5 our patches)

3
more fixes filed (DuckDB, Turso, DataFusion)

36.7%
backend lines (random 22.2%)

Take-away
Valid SQL is the easy part; knowing the right answer is the hard part. Typed trees make sound rewrites possible, measured precision shows where an oracle is still wrong, and the same core carries over to other engines.

Thank you. Questions?

github.com/PASS-IITG/cs5187-term-project-blazing-bhuvi

generalized: github.com/v28k/cs5187-term-project (branch `generalized`)